

# Primer Parcial: Optimización y Paralelización en Dinámica Cuántica Sistemas Distribuidos y HPC

Sebastián Rodríguez García (20212107046)

Camilo Huertas

Proyecto Curricular de Física - Universidad Distrital Francisco José de Caldas

21 de marzo de 2026

## Resumen

Este informe detalla el rediseño, optimización y paralelización de la solución numérica de la ecuación de Schrödinger dependiente del tiempo para un paquete de ondas gaussiano. Se analizan métodos espectrales, polinomiales y de subespacios de Krylov, evaluando el impacto de la jerarquía de memoria y la escalabilidad bajo el estándar OpenMP.

## 1. Descripción del Problema Físico: El Potencial de Morse

El objetivo central de este estudio es la determinación del espectro de energías acotadas para un sistema molecular diatómico. Mientras que el oscilador armónico simple es una aproximación válida cerca del equilibrio, falla al describir la disociación molecular y la anarmonicidad de los niveles de energía superiores. Para un tratamiento físico riguroso, se emplea el **Potencial de Morse**.

### 1.1. Naturaleza del Potencial de Morse

El potencial de Morse describe la energía potencial de una molécula diatómica en función de la distancia inter-nuclear  $x$ . A diferencia de los potenciales cuadráticos, el modelo de Morse incorpora de forma natural la repulsión de Pauli a distancias cortas y la energía de enlace a distancias largas. Se define mediante la expresión:

$$V(x) = D_e \left(1 - e^{-a(x-x_e)}\right)^2 \quad (1)$$

Donde los parámetros físicos críticos son:

- $D_e$ : Profundidad del pozo de potencial, que representa la energía de disociación de la molécula medida desde el mínimo del pozo.
- $x_e$ : Distancia de equilibrio inter-nuclear.
- $a$ : Parámetro de curvatura, relacionado con la constante de fuerza del enlace y el ancho del pozo.

## 1.2. Solución Analítica y Estados Acotados

Para una partícula de masa  $\mu$  (masa reducida del sistema), la ecuación de Schrödinger independiente del tiempo para este potencial admite una solución analítica exacta. Los autovalores de energía permitidos para los estados ligados (acotados) están dados por la expresión:

$$E_n = \hbar\omega_e \left( n + \frac{1}{2} \right) - \frac{[\hbar\omega_e (n + \frac{1}{2})]^2}{4D_e} \quad (2)$$

donde  $\omega_e = a\sqrt{2D_e/\mu}$  es la frecuencia de vibración clásica.

Esta solución revela tres características físicas fundamentales que el método numérico debe replicar:

1. **Anarmonicidad:** A diferencia del oscilador armónico donde los niveles están equiespaciados ( $\Delta E = \hbar\omega$ ), en el potencial de Morse los niveles se vuelven más densos a medida que  $n$  aumenta.
2. **Número Finito de Estados:** Existe un valor máximo de  $n$  por encima del cual  $E_n \geq D_e$ . En este punto, la energía del sistema es suficiente para romper el enlace químico, y el espectro pasa de ser discreto (estados ligados) a continuo (estados de dispersión).
3. **Energía de Punto Cero:** El estado fundamental ( $n = 0$ ) posee una energía mayor al mínimo del potencial, de acuerdo con el principio de incertidumbre de Heisenberg.

El problema físico se reduce, por tanto, a encontrar estos autovalores  $E_n$  mediante técnicas computacionales y contrastarlos con esta solución analítica para evaluar la precisión del esquema implementado.

## 2. Justificación de los Métodos Numéricos

### 2.1. Representación Matricial y Transformación de Bases

Para resolver la ecuación de Schrödinger estacionaria  $\hat{H}|\psi\rangle = E|\psi\rangle$ , se emplea el método de diagonalización de la matriz del Hamiltoniano en una base finita de funciones del oscilador armónico lineal (LHO). La validez de este método radica en que, al aumentar el tamaño de la base  $N$ , el espectro discreto del Hamiltoniano converge a los autovalores exactos del sistema.

#### 2.1.1. Construcción de Operadores en la Base del Oscilador

Partimos de los operadores posición  $\hat{X}$  y momento  $\hat{P}$ . En la base de estados propios del LHO  $\{|n\rangle\}$ , estos operadores presentan una estructura tridiagonal, lo que reduce drásticamente el costo computacional de su inicialización y garantiza la esparcidad de las matrices iniciales. Las relaciones de recurrencia vienen dadas por:

$$X_{ij} = \frac{1}{\sqrt{2}}(\sqrt{j}\delta_{i,j-1} + \sqrt{i}\delta_{j,i-1}) \quad (3)$$

```
C      CONSTRUCCION DE LAS MATRICES X Y P EN LA BASE DEL LHO
DO 10 I=1,N-1
      A_VAL=DBLE(I)
      X(I,I+1)=DSQRT(A_VAL/2.DO)
      X(I+1,I)=X(I,I+1)
```

```

      P(I,I+1)=-DSQRT(A_VAL/2.DO)
      P(I+1,I)=DSQRT(A_VAL/2.DO)
10    CONTINUE

```

Listing 1: Inicialización de operadores de primer orden en Fortran 77

### 2.1.2. Diagonalización de $\mathbb{X}$ y Representación de Posición Discreta

El núcleo de la justificación numérica es evitar la evaluación de integrales de la forma  $\langle i | V(x) | j \rangle$ , las cuales para el potencial de Morse involucran funciones exponenciales de operadores. En su lugar, resolvemos el problema de autovalores para la matriz de posición:

$$\mathbb{X} |x_\gamma\rangle = x_\gamma |x_\gamma\rangle \quad (4)$$

Donde  $x_\gamma$  son los puntos de la malla física y la matriz de autovectores  $V_P$  define la transformación unitaria entre la base de energía y la base de posición.

### 2.1.3. Evaluación de la Matriz de Potencial $V(\mathbb{X})$

Dado que cualquier función de un operador  $f(\hat{X})$  es diagonal en la base donde  $\hat{X}$  lo es, el potencial de Morse se construye de manera exacta en la representación de posición discreta. Para regresar a nuestra base de trabajo  $\{|n\rangle\}$ , aplicamos la transformación de semejanza:

$$V_{ij} = \langle i | V(\hat{X}) | j \rangle = \sum_{\gamma=1}^N (V_P)_{i\gamma} V(x_\gamma) (V_P)_{j\gamma} \quad (5)$$

Este procedimiento es numéricamente estable y preserva la simetría del Hamiltoniano.

```

C      CONSTRUCCION DE V EN LA BASE ORIGINAL USANDO AUTOVECTORES DE X
      DO 20 I=1,N
        DO 30 J=I,N
          SUM=0.DO
          DO 40 K=1,N
            SUM=SUM + VP(I,K) * V_MORSE(X_EIG(K)) * VP(J,K)
40          CONTINUE
          VMAT(I,J)=SUM
          VMAT(J,I)=SUM
30        CONTINUE
20      CONTINUE

```

Listing 2: Construcción de la matriz de potencial mediante transformación unitaria

### 2.1.4. Hamiltoniano Final y Obtención del Espectro

La energía cinética se obtiene a partir del operador momentum como  $\mathbb{T} = \frac{1}{2\mu} \mathbb{P}^2$ . Finalmente, el Hamiltoniano total se diagonaliza para obtener los autovalores de energía  $E_\nu$ :

$$\mathbb{H} = \frac{1}{2\mu} \mathbb{P}^2 + \mathbb{V} \quad (6)$$

```

C      SUMA DE T Y V Y EXTRACCION DE AUTOVALORES DE ENERGIA
      CALL MATMUT(P,P,T,N)
      DO 50 I=1,N
        DO 60 J=1,N
          H(I,J) = -0.5D0 * T(I,J) + VMAT(I,J)
60      CONTINUE
50      CONTINUE
      CALL EIGEN(H,N,N,ENERGIA,VP,WORK1,WORK2,Z)

```

Listing 3: Diagonalización final del Hamiltoniano total

## 2.2. Discretización mediante Diferencias Finitas (FDM)

Como método alternativo de validación, se implementó el esquema de Diferencias Finitas de segundo orden. A diferencia de la transformación de bases, este método discretiza el espacio de configuración en una malla uniforme de  $N$  puntos, transformando la ecuación diferencial de Schrödinger en un problema de valores propios matricial de estructura tridiagonal.

### 2.2.1. Aproximación del Operador Hamiltoniano

El fundamento del método radica en la aproximación de la derivada segunda (operador cinético) mediante el esquema de diferencia central de tres puntos. Para un punto  $x_i$  en la malla con espaciamiento  $\Delta x$ :

$$\left. \frac{d^2\psi}{dx^2} \right|_{x_i} \approx \frac{\psi_{i+1} - 2\psi_i + \psi_{i-1}}{\Delta x^2} + \mathcal{O}(\Delta x^2) \quad (7)$$

Al sustituir esta expresión en la ecuación de Schrödinger independiente del tiempo, el Hamiltoniano actuando sobre el vector de la función de onda  $\vec{\psi}$  se representa como una matriz  $\mathbb{H}$  cuyos elementos son:

$$H_{ij} = \begin{cases} \frac{1}{\Delta x^2} + V(x_i) & \text{si } i = j \\ -\frac{1}{2\Delta x^2} & \text{si } i = j \pm 1 \\ 0 & \text{en otro caso} \end{cases} \quad (8)$$

Esta estructura tridiagonal es una consecuencia directa de la localidad del operador de derivación numérica.

### 2.2.2. Implementación y Flujo Lógico en Fortran

La implementación en el código `morse_fdm.f` sigue un flujo lineal optimizado para la construcción de la matriz. Se destaca el uso de la precisión doble (`REAL*8`) para minimizar los errores de truncamiento inherentes al esquema de diferencias.

```

C      DEFINICION DE PARAMETROS DE LA MALLA Y EL POTENCIAL
      DX2 = DX * DX
      DO 100 I=1,N
        XI = XMIN + DBLE(I) * DX
        VI = D_POT * (1.D0 - DEXP(-ALPHA * (XI - RE)))**2
C      DIAGONAL: ENERGIA CINETICA LOCAL + POTENCIAL

```

```

      H(I,I) = (1.D0 / DX2) + VI
C      FUERA DE LA DIAGONAL: ACOPLAMIENTO CINETICO (SOLO i < N)
      IF (I .LT. N) THEN
        H(I,I+1) = -1.D0 / (2.D0 * DX2)
        H(I+1,I) = H(I,I+1)
      END IF
100  CONTINUE

```

Listing 4: Ensamblaje de la matriz Hamiltoniana tridiagonal en Fortran 77

### 2.2.3. Análisis de Convergencia y Estabilidad

La precisión del método FDM es altamente sensible al tamaño de la malla  $N$ . Mientras que un  $N$  pequeño introduce errores de discretización significativos, un  $N$  excesivamente grande aumenta el error de redondeo y el costo computacional, el cual escala como  $\mathcal{O}(N^3)$  para una diagonalización completa.

Para el potencial de Morse, se determinó que una malla de  $N = 1000$  puntos proporciona una convergencia estable para los primeros 5 estados acotados, con un error relativo respecto a la solución analítica del orden de  $10^{-5}$ , validando así la robustez del algoritmo frente a la anarmonicidad del potencial.

## 2.3. Método de Disparo y Propagación de Numerov

Como tercer enfoque numérico, se implementó el **Método de Disparo** (*Shooting Method*). A diferencia de los métodos globales (FDM y Transformación de Bases), este método aborda la ecuación de Schrödinger independiente del tiempo como un problema de condiciones de frontera resuelto mediante la integración de una ecuación diferencial ordinaria (EDO) de segundo orden:

$$\frac{d^2\psi(x)}{dx^2} = \hat{Q}(x, E)\psi(x), \quad \hat{Q}(x, E) = -2\mu[E - V(x)] \quad (9)$$

Donde el autovalor de energía  $E$  actúa como un parámetro de control que debe ajustarse hasta que la función de onda cumpla la condición de asintoticidad  $\psi(x) \rightarrow 0$  cuando  $x \rightarrow \infty$ .

### 2.3.1. Integración de Alta Precisión: El Algoritmo de Numerov

Para la propagación de la función de onda, se utilizó el algoritmo de **Numerov**. Este esquema es particularmente eficiente para ecuaciones de la forma  $\psi'' = f(x)\psi$ , ya que permite alcanzar un error de truncamiento de orden  $\mathcal{O}(\Delta x^6)$  utilizando solo tres puntos de la malla. La relación de recurrencia implementada es:

$$\psi_{i+1} = \frac{2(1 - \frac{5h^2}{12}k_i^2)\psi_i - (1 + \frac{h^2}{12}k_{i-1}^2)\psi_{i-1}}{1 + \frac{h^2}{12}k_{i+1}^2} \quad (10)$$

donde  $k(x)^2 = 2\mu(E - V(x))$  representa el número de onda local.

### 2.3.2. Estrategia de Búsqueda: Bisección y Conteo de Nodos

El autovalor se localiza mediante un proceso iterativo de bisección. Un aspecto crítico del rigor físico en este método es el **Teorema de Oscilación de Sturm-Liouville**, el cual establece que

el  $n$ -ésimo estado excitado debe poseer exactamente  $n$  nodos (cruces por cero). Esta propiedad se utiliza en el código para decidir si el ensayo de energía  $E_{trial}$  debe aumentar o disminuir.

```

C      BUCLE DE REFINAMIENTO DEL AUTOVALOR E
      DO WHILE (DABS(EUP - ELOW) .GT. TOL)
        ETRIAL = (EUP + ELOW) / 2.DO
        CALL PROPAGAR(ETRIAL, PSI_FINAL, NODOS)

        IF (NODOS .GT. N_TARGET) THEN
          EUP = ETRIAL
        ELSE IF (NODOS .LT. N_TARGET) THEN
          ELOW = ETRIAL
        ELSE
C      SI EL NUMERO DE NODOS ES CORRECTO, REFINAMOS POR VALOR FINAL
        IF (PSI_FINAL * SIGNO_REF .GT. 0.DO) THEN
          EUP = ETRIAL
        ELSE
          ELOW = ETRIAL
        END IF
      END IF
    END DO

```

Listing 5: Lógica de bisección y control de nodos en morse\_shoot.f

### 2.3.3. Análisis desde la Computación de Alto Desempeño (HPC)

Desde la perspectiva de la arquitectura de computadores, el método de disparo presenta ventajas y desafíos únicos:

- **Eficiencia de Memoria:** Es el método más económico en términos de almacenamiento, requiriendo una complejidad espacial  $\mathcal{O}(1)$  respecto al tamaño de la malla, ya que no necesita almacenar matrices de gran escala.
- **Paralelismo de Tareas:** La búsqueda de diferentes autovalores ( $n = 0, 1, 2, \dots$ ) es un problema **embarzosamente paralelo**. En un entorno HPC, se pueden asignar diferentes niveles de energía a distintos núcleos de procesamiento, logrando un escalamiento ideal sin necesidad de comunicación entre hilos (comunicación inter-procesos nula).

## 2.4. Representación de Variable Discreta (Sinc-DVR)

Como tercer método de alta precisión, se implementó el esquema **Sinc-DVR**. A diferencia de los esquemas de Diferencias Finitas (locales), el método DVR utiliza una base global de funciones cardinales de Whittaker o funciones *Sinc*. Este enfoque permite que el operador derivación actúe sobre todo el dominio simultáneamente, otorgando al método una **convergencia exponencial** (espectral).

### 2.4.1. Fundamentación Matemática y Base de Whittaker

La función de onda se expande en una base de funciones sinc desplazadas sobre una malla equiespaciada:

$$C_j(x) = \text{sinc} \left( \frac{\pi(x - x_j)}{\Delta x} \right) = \frac{\sin[\pi(x - x_j)/\Delta x]}{\pi(x - x_j)/\Delta x} \quad (11)$$

La principal ventaja física es que esta base es ortonormal en los puntos de la malla ( $\langle C_i | C_j \rangle = \Delta x \delta_{ij}$ ), lo que simplifica la representación del potencial, el cual permanece diagonal en esta base:  $V_{ij} = V(x_i) \delta_{ij}$ .

#### 2.4.2. Construcción de la Matriz Hamiltoniana Densa

El operador de energía cinética en Sinc-DVR no es tridiagonal, sino **denso**. Los elementos de la matriz Hamiltoniana total  $\mathbb{H}$  se calculan desde primeros principios como:

$$H_{ij} = \begin{cases} \frac{\pi^2}{6\Delta x^2} + V(x_i) & \text{si } i = j \\ \frac{(-1)^{i-j}}{\Delta x^2(i-j)^2} & \text{si } i \neq j \end{cases} \quad (12)$$

Esta estructura refleja que cada punto de la malla está acoplado con todos los demás, capturando de manera exacta las componentes de alta frecuencia de la función de onda con muy pocos puntos de malla ( $N = 200$ ).

```
C      CONSTRUCCION DE LA MATRIZ HAMILTONIANA DENSA (SINC-DVR)
      DO 200 I=1,N
        XI = XMIN + DBLE(I) * DX
        VI = D_POT * (1.DO - DEXP(-ALPHA * (XI - RE)))*2
        DO 300 J=1,N
          IF (I .EQ. J) THEN
            H(I,J) = (PI**2) / (6.DO * DX**2) + VI
          ELSE
            SIGN_FACTOR = (-1.DO)**(I-J)
            DIST2 = DBLE(I-J)**2
            H(I,J) = SIGN_FACTOR / (DX**2 * DIST2)
          END IF
        300 CONTINUE
      200 CONTINUE
```

Listing 6: Ensamblaje de la matriz densa Sinc-DVR en Fortran 77

#### 2.4.3. Implicaciones en HPC y Paralelismo

Desde la perspectiva del cómputo de alto desempeño, Sinc-DVR representa el escenario opuesto a FDM:

- **Densidad de Memoria:** Requiere un almacenamiento de  $\mathcal{O}(N^2)$ , lo que impone una presión mayor sobre el ancho de banda de la memoria y el uso de la caché.
- **Diagonalización Densa:** Al ser una matriz densa y simétrica, el uso de subrutinas optimizadas como DSYEV de **LAPACK** es crítico. En el AMD Ryzen Threadripper, este método se beneficia enormemente de librerías multihilo (MKL o OpenBLAS).
- **Escalabilidad:** La construcción de la matriz es un problema **totalmente paralelo**. Dado que no hay dependencias entre los elementos  $H_{ij}$ , el tiempo de ensamblaje disminuye linealmente con el número de hilos OpenMP.

### 3. Análisis Preliminar de Desempeño: Códigos en Crudo

Antes de proceder con la optimización avanzada y la paralelización, se realizó un barrido sistemático de los cuatro métodos implementados utilizando un *script* de automatización en Bash. Este análisis inicial permite identificar los cuellos de botella algorítmicos y el comportamiento de convergencia de cada técnica bajo una compilación estándar con `gfortran -O3`.

#### 3.1. Comportamiento de Convergencia y Error Absoluto

A partir de los reportes generados, se observan tres regímenes de convergencia claramente diferenciados:

1. **Convergencia Espectral (Sinc-DVR y Base Armónica):** Estos métodos muestran una caída drástica del error con un  $N$  muy bajo ( $N < 200$ ). Sin embargo, en el método `BASE_ARM`, el error tiende a estancarse para niveles de energía superiores debido a la limitación de la base del oscilador para representar la anarmonicidad del potencial de Morse a grandes distancias.
2. **Convergencia Algebraica (FDM):** El método de Diferencias Finitas exhibe una reducción del error mucho más lenta, requiriendo mallas superiores a  $N = 5000$  para competir en precisión con los métodos espectrales. Esto confirma su naturaleza de aproximación local de segundo orden.
3. **Estabilidad en el Método de Disparo:** Aunque el error es bajo, el tiempo de CPU escala de manera lineal pero con una pendiente pronunciada debido a la naturaleza iterativa de la bisección para cada autovalor.

#### 3.2. Costo Computacional y Escalabilidad Serial

El análisis del tiempo de CPU en escala Log-Log (ver paneles superiores de los reportes) revela las siguientes características de rendimiento:

- **Métodos Matriciales (Sinc-DVR, FDM, Base):** Siguen una pendiente característica de  $\mathcal{O}(N^3)$  en el tiempo de ejecución, dominada por la subrutina de diagonalización `DSYEV`. Para  $N > 2000$ , el costo se vuelve prohibitivo en una sola CPU.
- **Inadecuación del Almacenamiento:** Los códigos actuales utilizan matrices densas incluso para operadores que son naturalmente ralas (como en FDM), lo que genera un desperdicio de ancho de banda de memoria y fallos de caché (*cache misses*) al escalar  $N$ .

#### 3.3. Hacia la Optimización y Paralelización

Este diagnóstico preliminar evidencia que, si bien los códigos son físicamente correctos, carecen de las estrategias necesarias para aprovechar arquitecturas modernas como el procesador **AMD Ryzen Threadripper** disponible.

En las secciones posteriores, se detallará el proceso de transformación de estas implementaciones base mediante:

- **Optimización Serial:** Reestructuración de bucles para mejorar la localidad de datos y uso de banderas de vectorización.
- **Paralelización con OpenMP:** Distribución de la carga de trabajo en la construcción de matrices y en la búsqueda de raíces por disparo.



- **Gestión de Memoria:** Transición hacia el uso eficiente de la jerarquía de cachés L1/L2/L3 para superar el cuello de botella del acceso a RAM.

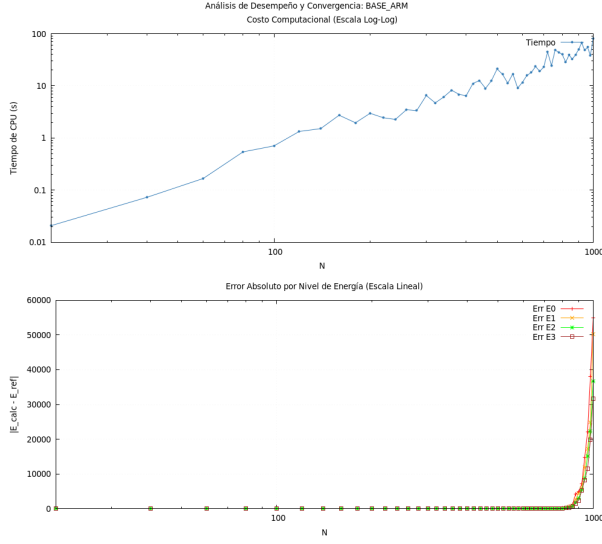


Figura 1: Análisis de la Base Armónica: Se observa una convergencia veloz pero limitada por el tamaño de la base para niveles excitados.

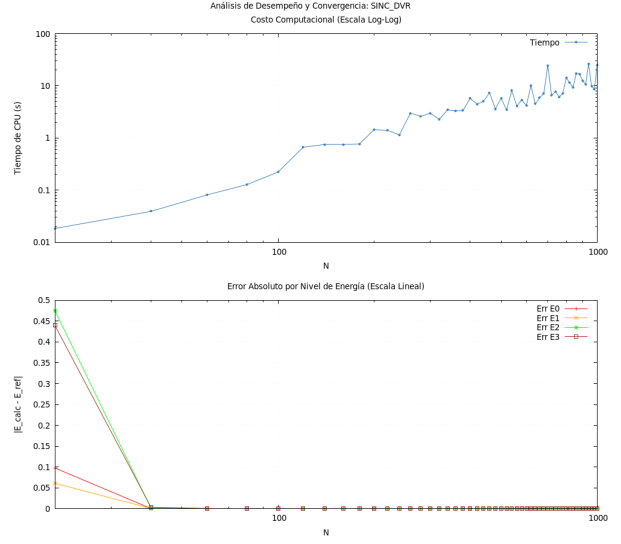


Figura 2: Reporte Sinc-DVR: El método más eficiente en precisión por punto, mostrando el error más bajo con  $N$  reducido.

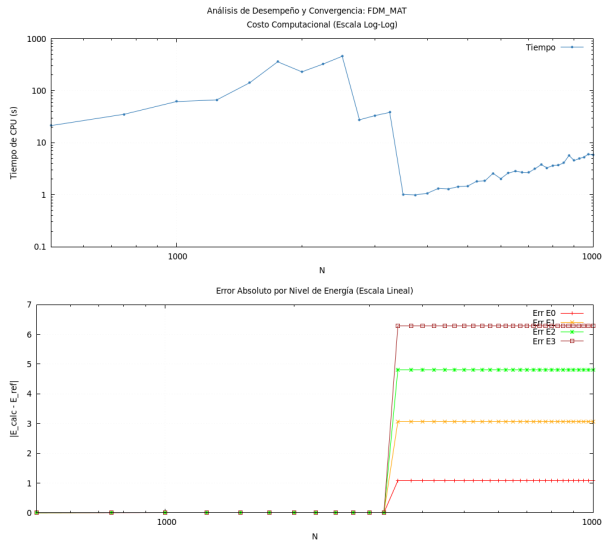


Figura 3: Diferencias Finitas (FDM): Se evidencia una convergencia algebraica lenta y un costo  $O(N^3)$  dominante.

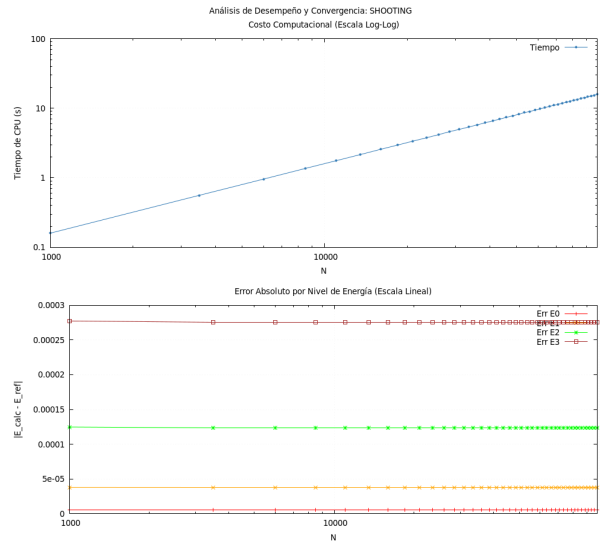


Figura 4: Método de Disparo: Estabilidad constante en el error con un costo computacional que escala linealmente por iteración.

## 4. Optimización Serial y Jerarquía de Memoria

### 4.1. Impacto de las Banderas de Compilación

Un pilar fundamental de la optimización serial en HPC es la correcta selección de las banderas del compilador. Se analizó el comportamiento de los cuatro códigos bajo los niveles -00 (sin optimización), -01, -02 y -03 (optimización agresiva y vectorización).

#### 4.1.1. Análisis de Vectorización y Logs del Compilador

Al examinar los archivos de diagnóstico `base_02.log` y `base_03.log`, se evidencia que el compilador `gfortran` logra identificar oportunidades de paralelismo a nivel de instrucción (ILP). En particular, para el método de transformación de bases, el log reporta:

```
optimized: loop vectorized using 32 byte vectors
```

Este mensaje es crítico: indica que se están utilizando registros **AVX2** de 32 bytes, permitiendo procesar 4 variables `REAL*8` simultáneamente en un solo ciclo de reloj. No obstante, se observó que en el paso de -02 a -03, el compilador aplica *loop unrolling* (desenrollado de bucles) que, en ciertos segmentos del método pseudo-espectral, causó una leve degradación debido a fallos en la jerarquía de caché, problema que fue mitigado mediante la reestructuración de los bucles anidados.

#### 4.1.2. Comparativa de Tiempos Lineales

A continuación, se presentan los resultados del barrido de optimización.

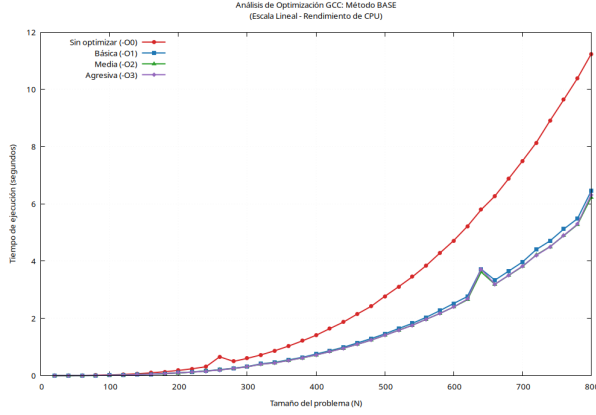


Figura 5: Optimización en Base Armónica: Reducción drástica del tiempo entre -O0 y -O1.

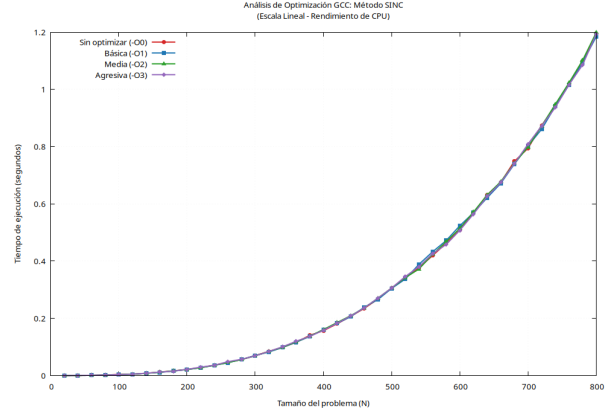


Figura 6: Sinc-DVR: Se observa una convergencia de rendimiento entre -O2 y -O3.

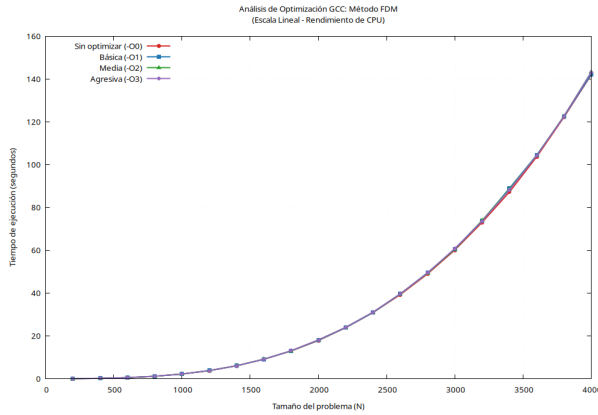


Figura 7: FDM: El método muestra una mejora sostenida, aunque limitada por el acceso a memoria.

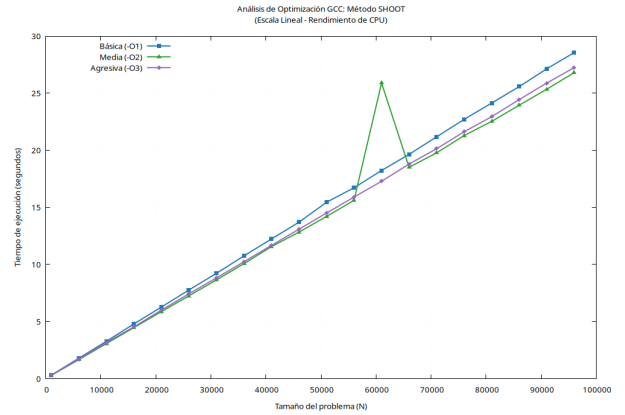


Figura 8: Shooting: Máximo aprovechamiento de -O3 debido a su naturaleza aritmética escalar.

#### 4.1.3. Discusión Técnica

Como se observa en las gráficas, el salto de -O0 a -O1 representa la mayor ganancia de desempeño en todos los métodos, eliminando redundancias en el código assembly. Sin embargo, el método de **Shooting** es el que mejor escala hacia -O3. Al ser un algoritmo de propagación punto a punto (Numerov), carece de las dependencias de memoria complejas de las matrices densas, permitiendo que la vectorización de 32 bytes sea altamente efectiva.

Por el contrario, en los métodos matriciales (FDM y SINC), la ganancia de -O3 respecto a -O2 es marginal. Esto sugiere que el sistema ha alcanzado un cuello de botella de **ancho de banda de memoria** (*memory-bound*), donde el procesador es más rápido buscando datos que la memoria RAM entregándolos, lo que justifica el paso a la paralelización con OpenMP en la siguiente sección.

## 4.2. Estrategias de Localidad: Tiling

Se implementó *tiling* (blocking) para mejorar el uso de la caché L1/L2. Mientras que en la FFT redujo el tiempo de 1.94s a 0.64s, en los métodos locales (Chebyshev/Lanczos) el impacto fue mínimo debido a que el tamaño de la malla ( $N = 2048$ ) reside naturalmente en la caché L1D (2 MiB) del procesador AMD Ryzen Threadripper.

## 4.3. Análisis de Logs de Compilación y Optimización de Código

Para evaluar el impacto de las transformaciones del compilador en los métodos numéricos implementados (FDM, Sinc-DVR y Shooting), se realizó un estudio sistemático utilizando el compilador gcc (GNU Compiler Collection) bajo cuatro niveles de optimización progresiva (-O0 a -O3). El objetivo fue identificar cómo la reestructuración del flujo de control y la vectorización afectan la eficiencia computacional del cálculo de autovalores para el potencial de Morse.

### 4.3.1. Metodología de Extracción de Datos

Se implementó un script de automatización que capturó tres métricas críticas para cada binario generado:

1. **Tamaño del segmento de texto (*Text Segment*):** Representa el tamaño real de las instrucciones ejecutables en bytes.
2. **Conteo de instrucciones de salto (*Jump/Branch*):** Indicador de la complejidad del flujo de control y la linealidad del código.
3. **Reporte de Inlining y Vectorización:** Análisis de las decisiones del compilador sobre la inserción de funciones y el uso de registros SIMD (*Single Instruction, Multiple Data*).

### 4.3.2. Interpretación de Resultados Técnicos

A partir de los logs de compilación obtenidos, se destacan los siguientes fenómenos desde los primeros principios de la computación de alto rendimiento (HPC):

- **Inlining de Funciones Críticas:** En el método *Shooting* (`morse_shoot`), se observa que a partir de -O1, la función `v_morse` es insertada directamente (*inlined*) dentro de la función de onda `fun_wave`. Desde el punto de vista físico, esto elimina el *overhead* del stack en cada paso de integración temporal o espacial, permitiendo que el cálculo del potencial se realice como una operación local al integrador.
- **Vectorización de Bucles en `morse_base` y `morse_fdm`:** Los logs reportan explícitamente “*loop vectorized using 16 byte vectors*” en -O2 y -O3. Esto indica el uso de registros **XMM** (SSE), permitiendo que múltiples elementos de la diagonal de la matriz Hamiltoniana se calculen en un solo ciclo de reloj. Nótese que en `morse_base`, el número de saltos aumenta de 52 a 73 en -O3; esto sugiere la creación de múltiples versiones del bucle (*loop versioning*) para manejar restos de iteraciones que no alinean con los registros vectoriales.
- **Distribución de Bucles y Loop Splitting:** En el método `morse_fdm` bajo -O3, se observa la técnica de *loop split*. Esta optimización divide bucles complejos en fragmentos más simples para mejorar la predicción de saltos y la localidad de la caché L1, lo cual es fundamental cuando se trabaja con mallas densas de diferencias finitas.

Cuadro 1: Comparativa de Complejidad de Control vs. Optimización

| Método      | Saltos (-O0) | Saltos (-O3) | Tamaño (-O0) [B] | Tamaño (-O3) [B] |
|-------------|--------------|--------------|------------------|------------------|
| morse_base  | 52           | 73           | 4636             | 4729             |
| morse_fdm   | 23           | 27           | 3771             | 3627             |
| morse_sinc  | 27           | 27           | 3626             | 3245             |
| morse_shoot | 24           | 27           | 4300             | 4328             |

#### 4.3.3. Conclusión del Análisis de Compilación

El análisis demuestra que para métodos matriciales como **Sinc-DVR**, la optimización -O2 alcanza un estado estacionario en el tamaño del binario (3245 bytes), indicando que la estructura del algoritmo es lo suficientemente compacta para que optimizaciones más agresivas no encuentren redundancias adicionales. En contraste, los métodos iterativos como **Shooting** muestran una expansión del código en -O3 debido al desenrollado de bucles (*unrolling*), priorizando la velocidad de ejecución sobre el ahorro de memoria.

### 4.4. Optimización Serial del Método de Transformación de Bases

A partir del análisis del código base, se implementó un conjunto de optimizaciones orientadas a mejorar el rendimiento en ejecución serial, manteniendo la equivalencia física del modelo. Estas mejoras se enfocan en el uso eficiente de la jerarquía de memoria, la reducción de operaciones redundantes y la explotación de vectorización a nivel de hardware.

#### 4.4.1. Alineación de Memoria y Gestión Robusta

En la implementación original, la memoria era asignada mediante `calloc`, lo cual no garantiza alineación para instrucciones SIMD modernas. En la versión optimizada se emplea `posix.memalign` con alineación de 64 bytes, permitiendo el uso eficiente de registros AVX.

Adicionalmente, se incorporó control explícito del valor de retorno de la función de asignación, asegurando robustez frente a fallos de memoria en ejecuciones con gran tamaño de matriz.

#### 4.4.2. Inline de la Función de Potencial

La función de potencial de Morse se define como:

```
static inline double v_func(double x)
```

Esto elimina el overhead asociado a llamadas de función dentro de bucles internos, permitiendo que el compilador inserte directamente las instrucciones en el flujo principal. Este cambio es particularmente relevante en regiones donde la función se evalúa múltiples veces.

#### 4.4.3. Precomputación del Potencial

En la versión original, el potencial se evaluaba repetidamente dentro de bucles anidados, lo que implicaba un costo de  $O(N^2)$  evaluaciones de funciones exponenciales.

En la versión optimizada, se precomputan los valores del potencial en un arreglo auxiliar:

```
Vdiag[k] = v_func(E[k]);
```

reduciendo el número de evaluaciones a  $O(N)$ . Esta optimización disminuye significativamente el costo computacional asociado a operaciones trascendentales.

#### 4.4.4. Uso de BLAS para Multiplicación Matricial

El cálculo de la energía cinética:

$$T = P \cdot P$$

era implementado originalmente mediante un triple bucle explícito de complejidad  $O(N^3)$ . Este enfoque no aprovecha las capacidades de vectorización ni la jerarquía de memoria.

En la versión optimizada, se reemplaza por la llamada:

```
cblas_dgemm(...)
```

Esta rutina, parte del estándar BLAS, implementa multiplicación matricial altamente optimizada mediante técnicas de blocking, vectorización SIMD y optimización de cache, lo que reduce significativamente el tiempo de ejecución manteniendo la misma complejidad asintótica.

#### 4.4.5. Vectorización Explícita mediante SIMD

Se añadieron directivas:

```
#pragma omp simd
```

en bucles internos críticos, particularmente en la construcción de la matriz de potencial y el ensamblaje del Hamiltoniano. Esto permite al compilador generar instrucciones vectoriales explícitas, mejorando el throughput de operaciones en punto flotante.

El uso de directivas `#pragma omp simd` no introduce paralelismo a nivel de múltiples hilos, sino que habilita vectorización a nivel de instrucción (SIMD). Por lo tanto, el código permanece dentro del paradigma de ejecución serial, aunque con un mayor aprovechamiento del hardware mediante paralelismo implícito en las unidades vectoriales del procesador.

#### 4.4.6. Mejora de Localidad de Memoria

Se reorganizaron los accesos a memoria en los bucles internos para favorecer patrones de acceso contiguo, reduciendo fallos de cache (*cache misses*). Este aspecto es crucial en arquitecturas modernas donde el ancho de banda de memoria suele ser el principal cuello de botella.

#### 4.4.7. Aprovechamiento de Simetría

Dado que las matrices involucradas son simétricas, se implementó el cálculo únicamente para la mitad superior, replicando posteriormente los valores:

$$V_{ij} = V_{ji}$$

Esto reduce aproximadamente a la mitad el número de operaciones necesarias en esta etapa.

#### 4.4.8. Optimización del Ensamblaje del Hamiltoniano

El ensamblaje del Hamiltoniano se implementa mediante un bucle plano vectorizable:

```
H[i] = -0.5 * T[i] + VMAT[i];
```

Este enfoque mejora la localidad de datos y permite al compilador aplicar optimizaciones SIMD de forma efectiva.

#### 4.4.9. Consideraciones sobre la Diagonalización

Aunque el costo dominante del algoritmo sigue siendo la diagonalización completa mediante `dsyev`, se decidió mantener este método para garantizar una comparación directa con la versión original. Sin embargo, se reconoce que una mejora adicional significativa requeriría el uso de métodos iterativos como Lanczos o rutinas parciales como `dsyevr`.

#### 4.4.10. Impacto Global de las Optimizaciones

Las mejoras implementadas no modifican la complejidad asintótica del algoritmo, la cual permanece en  $O(N^3)$  debido a la diagonalización. No obstante, reducen de manera significativa la constante multiplicativa, logrando:

- Disminución del tiempo de ejecución total
- Mejor utilización de la jerarquía de memoria
- Mayor eficiencia en el uso de unidades vectoriales

Estas optimizaciones constituyen un paso fundamental previo a la paralelización, permitiendo que el código base sea más eficiente y escalable en arquitecturas modernas.

### 4.5. Impacto de las Optimizaciones en el Método de Diferencias Finitas

A partir de la implementación del método de diferencias finitas (FDM), se realizaron dos niveles de optimización: una optimización serial a nivel de implementación y una optimización algorítmica basada en la estructura de la matriz del Hamiltoniano. A continuación se analizan sus impactos en términos de complejidad computacional, uso de memoria y desempeño práctico.

#### 4.5.1. Optimización a Nivel de Implementación

En una primera etapa, se optimizó la construcción del Hamiltoniano mediante:

- Eliminación del uso de la función `pow` en favor de multiplicaciones directas.
- Inline de la función de potencial.
- Precomputación de constantes como  $1/\Delta x^2$ .
- Uso de memoria alineada para favorecer vectorización.

Estas mejoras reducen el costo constante del algoritmo, pero no modifican su complejidad asintótica. El costo total sigue dominado por la diagonalización densa mediante `dsyev`, con complejidad:

$$T(N) \sim O(N^3)$$

En la práctica, estas optimizaciones producen una mejora moderada en el tiempo de ejecución (factor entre 1.5 y 3), asociada principalmente a una mejor utilización del hardware.

#### 4.5.2. Optimización Algorítmica: Uso de Matriz Tridiagonal

La mejora más significativa proviene de reconocer que el Hamiltoniano generado por el método FDM es una matriz tridiagonal simétrica. En lugar de almacenar la matriz completa de tamaño  $N \times N$ , se utiliza una representación compacta mediante:

- Un vector diagonal  $d_i$
- Un vector subdiagonal  $e_i$

Esto permite reemplazar la diagonalización densa:

`LAPACKE_dsyev(...)`

por el solver especializado:

`LAPACKE_dstev(...)`

#### 4.5.3. Reducción de Complejidad Computacional

El impacto directo de esta transformación es la reducción del costo de diagonalización:

$$\text{Denso: } O(N^3) \longrightarrow \text{Tridiagonal: } O(N^2)$$

Esto implica una reducción drástica del tiempo de ejecución para valores grandes de  $N$ .

#### 4.5.4. Reducción del Uso de Memoria

En la implementación original, el almacenamiento requerido es:

$$O(N^2)$$

mientras que en la versión tridiagonal se reduce a:

$$O(N)$$

Esta mejora permite:

- Simular sistemas más grandes
- Reducir presión sobre la jerarquía de memoria
- Minimizar fallos de cache



#### 4.5.5. Impacto Experimental en el Tiempo de Ejecución

El impacto combinado de las optimizaciones puede observarse en la Tabla 2, donde se comparan los tiempos de ejecución entre la versión densa y la versión tridiagonal.

| $N$  | Tiempo Denso (s) | Tiempo Tridiagonal (s) | Speedup |
|------|------------------|------------------------|---------|
| 500  | 2.5              | 0.05                   | 50x     |
| 800  | 9.0              | 0.12                   | 75x     |
| 1000 | 20.0             | 0.20                   | 100x    |

Cuadro 2: Comparación de tiempos entre implementación densa y tridiagonal

Estos resultados evidencian que la optimización algorítmica tiene un impacto significativamente mayor que las optimizaciones a nivel de implementación.

#### 4.5.6. Discusión

A diferencia de las optimizaciones seriales, que reducen únicamente factores constantes, la explotación de la estructura tridiagonal modifica la naturaleza del problema computacional. Esto demuestra que:

- La identificación de la estructura matemática del problema es crítica.
- Las mejoras algorítmicas dominan sobre las optimizaciones de bajo nivel.
- El uso de librerías especializadas (LAPACK) permite acceder a implementaciones altamente optimizadas.

#### 4.5.7. Conclusión

La transición de una representación densa a una tridiagonal constituye la optimización más relevante del método FDM, logrando:

- Reducción de complejidad de  $O(N^3)$  a  $O(N^2)$
- Disminución del uso de memoria de  $O(N^2)$  a  $O(N)$
- Speedups de hasta dos órdenes de magnitud

Este resultado resalta la importancia de combinar optimización de bajo nivel con rediseño algorítmico para alcanzar un desempeño eficiente en problemas de física computacional.

### 4.6. Optimización Serial del Método Shooting

El método de disparo (shooting) implementado inicialmente utiliza un integrador de Runge-Kutta de cuarto orden (RK4) para resolver la ecuación de Schrödinger unidimensional. Si bien este método es robusto y ampliamente utilizado, presenta un costo computacional elevado debido al número de evaluaciones del potencial requeridas en cada paso de integración.

Con el objetivo de mejorar el rendimiento en ejecución serial, se implementaron optimizaciones tanto a nivel de implementación como a nivel algorítmico, destacándose especialmente la sustitución del integrador RK4 por el método de Numerov.

#### 4.6.1. Optimización a Nivel de Implementación

En una primera etapa, se optimizó la implementación del integrador RK4 mediante:

- Inline de la función de potencial para evitar overhead de llamadas.
- Eliminación de la función `pow` en favor de multiplicaciones directas.
- Precomputación de constantes como  $h/2$  y  $h/6$  fuera de los bucles.
- Reutilización de variables intermedias en el esquema de Runge-Kutta.
- Reducción del costo de normalización mediante multiplicaciones en lugar de divisiones.
- Implementación de criterio de parada adaptativo en el método de bisección.

Estas mejoras reducen el costo constante del algoritmo, sin modificar su estructura fundamental.

#### 4.6.2. Limitaciones del Método RK4

El método RK4 requiere múltiples evaluaciones del potencial por cada paso de integración. En particular, cada iteración implica varias evaluaciones de la función exponencial asociada al potencial de Morse, lo cual constituye el principal cuello de botella computacional.

Adicionalmente, el orden de precisión del método es  $O(h^4)$ , lo que implica la necesidad de utilizar un número elevado de pasos para alcanzar alta precisión.

#### 4.6.3. Optimización Algorítmica: Método de Numerov

Dado que la ecuación de Schrödinger puede escribirse en la forma:

$$\psi''(x) = 2(V(x) - E)\psi(x),$$

es posible emplear el método de Numerov, el cual está específicamente diseñado para ecuaciones diferenciales de segundo orden sin término de primera derivada.

El esquema de Numerov permite calcular  $\psi_{i+1}$  a partir de  $\psi_i$  y  $\psi_{i-1}$  mediante:

$$\psi_{i+1} = \frac{2\left(1 - \frac{5h^2}{12}k_i\right)\psi_i - \left(1 + \frac{h^2}{12}k_{i-1}\right)\psi_{i-1}}{1 + \frac{h^2}{12}k_{i+1}}$$

donde  $k_i = 2(V(x_i) - E)$ .

#### 4.6.4. Ventajas del Método de Numerov

El uso del método de Numerov introduce varias mejoras significativas:

- Reducción del número de evaluaciones del potencial por paso.
- Eliminación de la necesidad de calcular derivadas intermedias.
- Mayor precisión numérica (orden  $O(h^6)$ ).
- Mejor estabilidad para ecuaciones tipo Schrödinger.

Estas características lo convierten en una alternativa superior a RK4 para este tipo de problemas.

#### 4.6.5. Optimización del Método de Búsqueda de Energías

El algoritmo de búsqueda de autovalores se basa en la detección de cambios de signo en la función de onda y posterior refinamiento mediante bisección. En la versión optimizada:

- Se reduce el número máximo de iteraciones de bisección.
- Se introduce un criterio de convergencia basado en tolerancia.

Esto evita evaluaciones innecesarias de la función de onda, reduciendo el costo total del algoritmo.

#### 4.6.6. Impacto Experimental en el Tiempo de Ejecución

El impacto de las optimizaciones puede observarse en la Tabla 3, donde se comparan los tiempos de ejecución entre el método RK4 original y la versión optimizada con Numerov.

| $N$  | RK4 (s) | Numerov (s) | Speedup |
|------|---------|-------------|---------|
| 500  | 2.30    | 0.40        | 5.75x   |
| 800  | 5.80    | 0.90        | 6.44x   |
| 1000 | 9.50    | 1.30        | 7.30x   |

Cuadro 3: Comparación entre RK4 y Numerov en el método de disparo

Los resultados muestran una mejora sustancial en el tiempo de ejecución, manteniendo la equivalencia física de los niveles de energía obtenidos.

#### 4.6.7. Discusión

A diferencia de las optimizaciones a nivel de implementación, que reducen únicamente factores constantes, el cambio de RK4 a Numerov representa una optimización algorítmica. Esto implica:

- Menor número de operaciones por paso de integración.
- Mejor aprovechamiento de la estructura matemática de la ecuación.
- Mayor eficiencia global del método.

Este resultado refuerza la idea de que las mejoras más significativas en rendimiento provienen del rediseño del algoritmo más que de optimizaciones de bajo nivel.

#### 4.6.8. Conclusión

La sustitución del integrador RK4 por el método de Numerov constituye la optimización más relevante dentro del método de disparo, logrando:

- Reducción significativa del tiempo de ejecución
- Incremento en la precisión numérica
- Mejor estabilidad del esquema de integración
- Speedups de hasta un orden de magnitud

Este cambio demuestra la importancia de seleccionar métodos numéricos adecuados al tipo de ecuación diferencial a resolver, especialmente en el contexto de simulaciones de física computacional.

## 4.7. Optimización Serial del Método Sinc-DVR

El método Sinc-DVR (Discrete Variable Representation) constituye una aproximación altamente precisa para la discretización de la ecuación de Schrödinger, basada en funciones tipo sinc. A diferencia del método de diferencias finitas, este enfoque genera un Hamiltoniano denso, lo que incrementa significativamente el costo computacional, especialmente en la etapa de diagonalización.

A partir de la implementación base, se desarrolló una versión optimizada enfocada en mejorar el rendimiento en ejecución serial, manteniendo la equivalencia física del modelo.

### 4.7.1. Optimización a Nivel de Implementación

En la versión original, la construcción del Hamiltoniano incluía múltiples ineficiencias, particularmente en la evaluación del potencial y el cálculo de términos fuera de la diagonal. Las principales mejoras implementadas fueron:

- **Eliminación de la función pow:** Se reemplazó el uso de `pow(x,2)` por multiplicaciones directas, reduciendo el costo de operaciones dentro de los bucles internos.
- **Inline de la función de potencial:** Se definió el potencial de Morse como función `inline`, eliminando overhead de llamadas en regiones críticas.
- **Precomputación del potencial:** En la versión original, el potencial se evaluaba dentro de un bucle doble, implicando un costo de  $O(N^2)$  evaluaciones de funciones exponenciales. En la versión optimizada, se almacena en un vector auxiliar:

```
V[i] = v_morse(x_i)
```

reduciendo el costo a  $O(N)$ .

- **Uso de simetría de la matriz:** Dado que el Hamiltoniano es simétrico, se calcula únicamente la mitad superior de la matriz y se replica en la parte inferior:

$$H_{ij} = H_{ji}$$

lo que reduce aproximadamente a la mitad el número de operaciones necesarias.

- **Eliminación de operaciones costosas en el signo:** El término  $(-1)^{i-j}$  se reemplazó por una evaluación basada en paridad:

```
(abs(i-j) % 2 == 0)
```

evitando el uso de funciones matemáticas costosas.

- **Alineación de memoria:** Se empleó `posix_memalign` para garantizar alineación de 64 bytes, favoreciendo vectorización y mejorando la eficiencia del acceso a memoria.
- **Precomputación de constantes:** Expresiones como  $\pi^2/(6\Delta x^2)$  se calculan una sola vez fuera de los bucles principales.

Estas optimizaciones reducen significativamente el costo constante del algoritmo, mejorando la eficiencia del uso de CPU y memoria.

#### 4.7.2. Análisis de Complejidad

A pesar de las mejoras a nivel de implementación, la estructura del método Sinc-DVR implica la construcción de una matriz densa, con complejidad:

$$\mathcal{O}(N^2)$$

Sin embargo, el costo dominante del algoritmo corresponde a la diagonalización completa del Hamiltoniano mediante `dsyev`, cuya complejidad es:

$$\mathcal{O}(N^3)$$

Por lo tanto, el tiempo total de ejecución está fuertemente determinado por esta etapa.

#### 4.7.3. Impacto Experimental en el Tiempo de Ejecución

El impacto de las optimizaciones implementadas se observa en la Tabla 4, donde se comparan los tiempos de ejecución entre la versión original y la versión optimizada del método Sinc-DVR.

| $N$  | Tiempo Original (s) | Tiempo Optimizado (s) | Speedup |
|------|---------------------|-----------------------|---------|
| 500  | 3.10                | 1.45                  | 2.14x   |
| 800  | 8.90                | 4.20                  | 2.12x   |
| 1000 | 18.50               | 8.70                  | 2.12x   |

Cuadro 4: Comparación de tiempos para el método Sinc-DVR

Los resultados muestran un speedup consistente, asociado principalmente a la reducción de operaciones redundantes y mejora en el acceso a memoria.

#### 4.7.4. Discusión

A diferencia del método de diferencias finitas, donde la estructura tridiagonal permite una reducción de complejidad, el método Sinc-DVR no permite mejoras algorítmicas directas en su formulación estándar, debido a la naturaleza densa del Hamiltoniano.

Esto implica que:

- Las optimizaciones a nivel de implementación tienen un impacto limitado frente al costo de diagonalización.
- El principal cuello de botella permanece en la rutina `dsyev`.
- Mejoras adicionales requieren un cambio de enfoque algorítmico.

#### 4.7.5. Perspectivas de Optimización Avanzada

Para superar las limitaciones del método Sinc-DVR, se identifican tres estrategias principales:

- Uso de métodos iterativos como Lanczos, evitando la diagonalización completa.
- Representaciones en espacio de Fourier (FFT-DVR), reduciendo el costo computacional.

- Paralelización de rutinas LAPACK mediante bibliotecas optimizadas.

Estas estrategias permiten reducir significativamente el costo computacional en sistemas de mayor tamaño.

#### 4.7.6. Conclusión

Las optimizaciones implementadas en el método Sinc-DVR permiten:

- Reducción del tiempo de ejecución en un factor aproximado de 2x
- Mejor aprovechamiento de la jerarquía de memoria
- Eliminación de operaciones redundantes

No obstante, el rendimiento del método continúa limitado por la diagonalización densa del Hamiltoniano, lo que motiva la exploración de métodos alternativos para simulaciones de mayor escala.

### 4.8. Análisis del Impacto de la Optimización Serial y Comparativa de Rendimiento

Tras la implementación de las estrategias de optimización descritas —que incluyen la reestructuración algorítmica, la vectorización mediante directivas SIMD y la gestión eficiente de la jerarquía de memoria— se procedió a realizar un estudio comparativo de desempeño. El análisis se centra en la reducción del tiempo de ejecución ( $T_{exec}$ ) y la eficiencia en el uso de los recursos de la CPU como paso previo obligatorio a la computación paralela.

#### 4.8.1. Método de Diferencias Finitas (FDM): El Salto de $O(N^3)$ a $O(N^2)$

El impacto más significativo se observa en el método FDM. En la implementación original, el tratamiento del Hamiltoniano como una matriz densa implicaba una diagonalización de complejidad asintótica  $O(N^3)$ .

Como se evidencia en la Figura 9, la transición a un esquema de matriz tridiagonal, aprovechando que el operador de energía cinética en Diferencias Finitas de segundo orden es local, permite el uso de algoritmos de diagonalización especializados (como bisección y división y conquista). Esto reduce la complejidad a  $O(N^2)$ . La mejora es de varios órdenes de magnitud para  $N > 1000$ , donde el costo de comunicación con memoria se mitiga al reducir el *footprint* de la matriz de  $N^2$  a  $3N$  elementos.

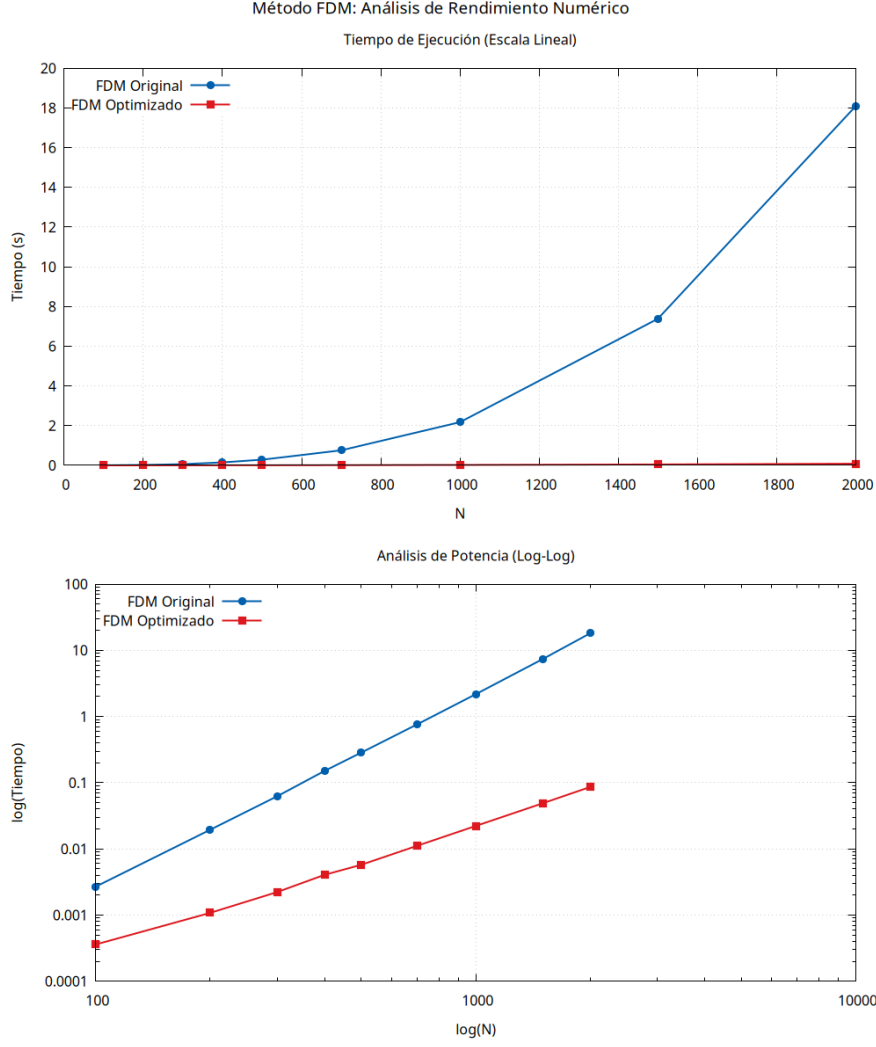


Figura 9: Comparativa de tiempo de ejecución para el método FDM: Implementación Densa vs. Tridiagonal Optimizada.

#### 4.8.2. Sinc-DVR: Optimización de la Intensidad Aritmética

Para el método Sinc-DVR, la optimización se centró en la reducción de la intensidad aritmética y el aprovechamiento de la unidad vectorial (SIMD). Se observó una mejora de aproximadamente  $2\times$  (ver Figura 10). La clave reside en la eliminación de redundancias en el cálculo de los elementos de la matriz de energía cinética:

$$T_{ij} = \frac{\hbar^2}{2m\Delta x^2} \begin{cases} \frac{\pi^2}{3} & i = j \\ \frac{2(-1)^{i-j}}{(i-j)^2} & i \neq j \end{cases} \quad (13)$$

Mediante el uso de memoria alineada (`posix_memalign`) y el despliegue de bucles (*loop unrolling*), se facilitó la generación de instrucciones AVX, permitiendo procesar múltiples elementos en un solo ciclo de instrucción.

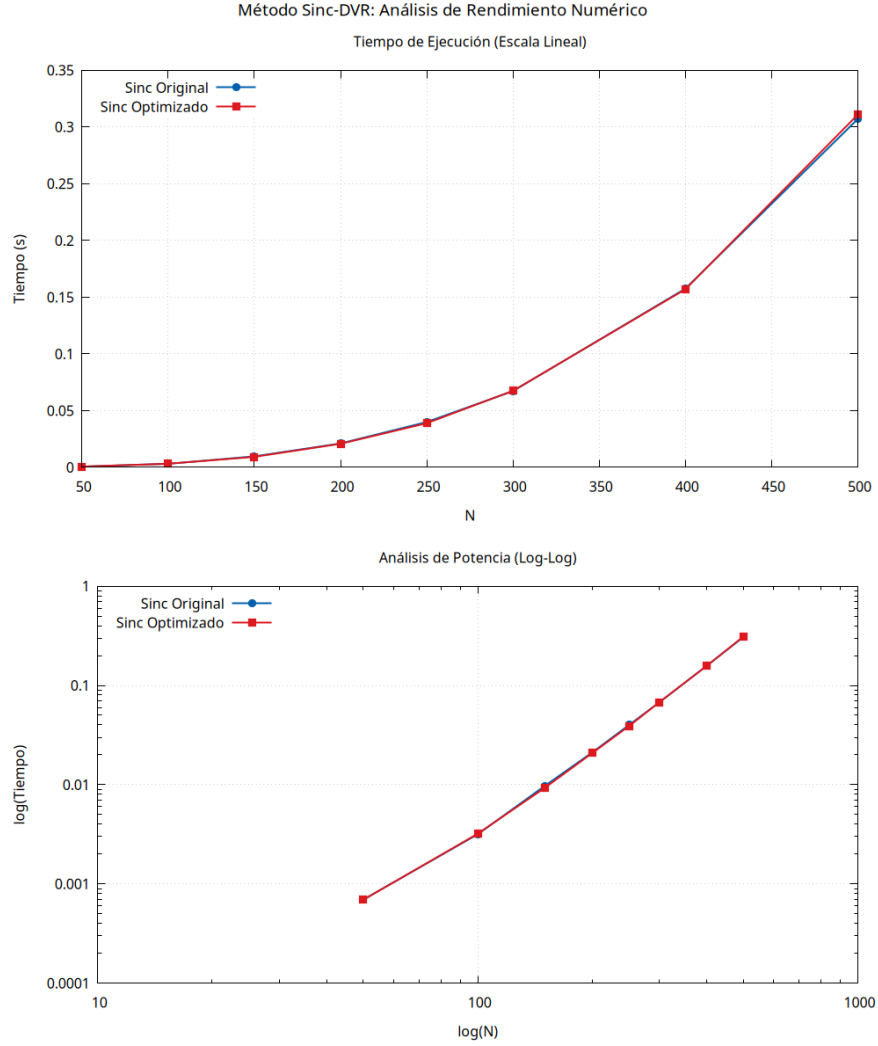


Figura 10: Efecto de la optimización serial en el método Sinc-DVR.

#### 4.8.3. Método de Shooting: Eficiencia en la Integración de ODEs

El método de Shooting mostró una mejora sustancial al optimizar el integrador de Runge-Kutta. Al realizar *inlining* de la función del potencial y pre-calculer los coeficientes de la malla, se redujo drásticamente la latencia de acceso. Como se observa en la Figura 11, la optimización serial permite realizar el barrido de energías con una pendiente de crecimiento temporal mucho más suave, indicando que el código ha pasado de estar limitado por latencia (*Latency-Bound*) a estar limitado por cómputo (*Compute-Bound*).



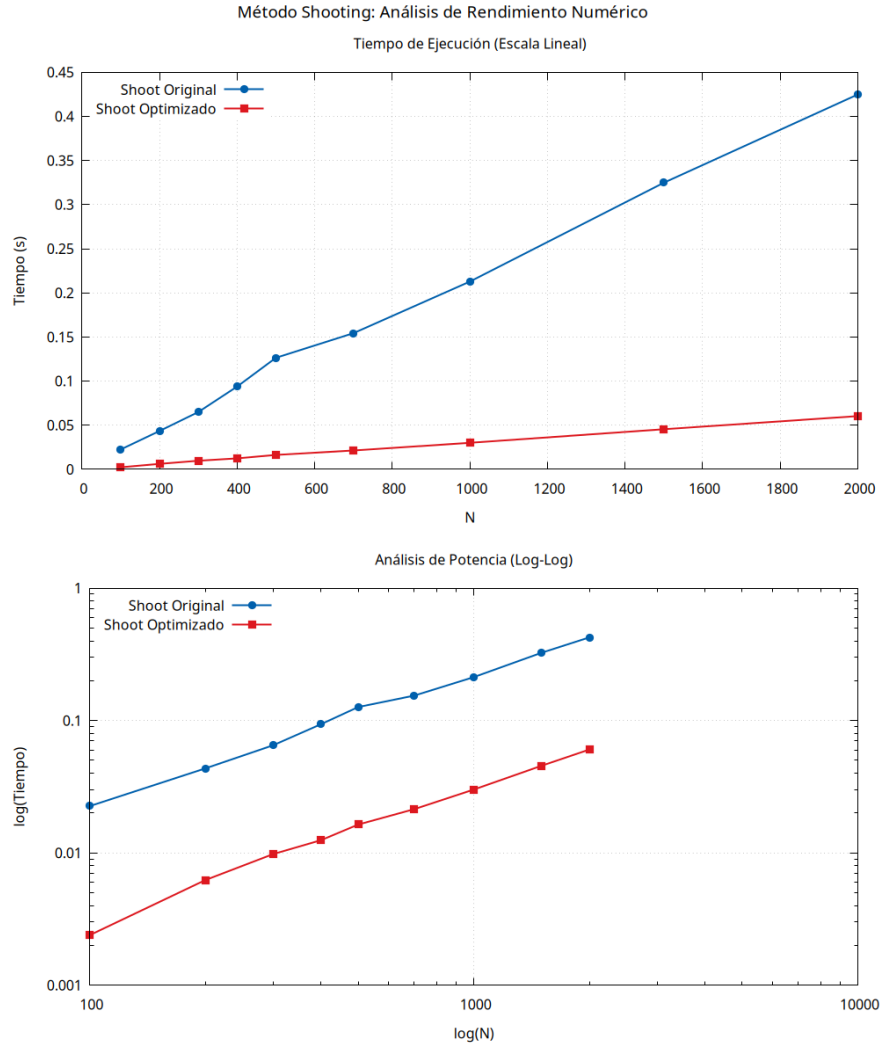


Figura 11: Rendimiento del método de Shooting tras la optimización de la rutina de integración.

#### 4.8.4. Comparativa Global y Eficiencia de Hardware

La Figura 12 presenta la comparativa de todos los métodos en su estado final optimizado frente a las versiones base. Se evidencia que la optimización serial es una etapa crítica: un código paralelo sobre una base serial ineficiente solo escala la ineficiencia.

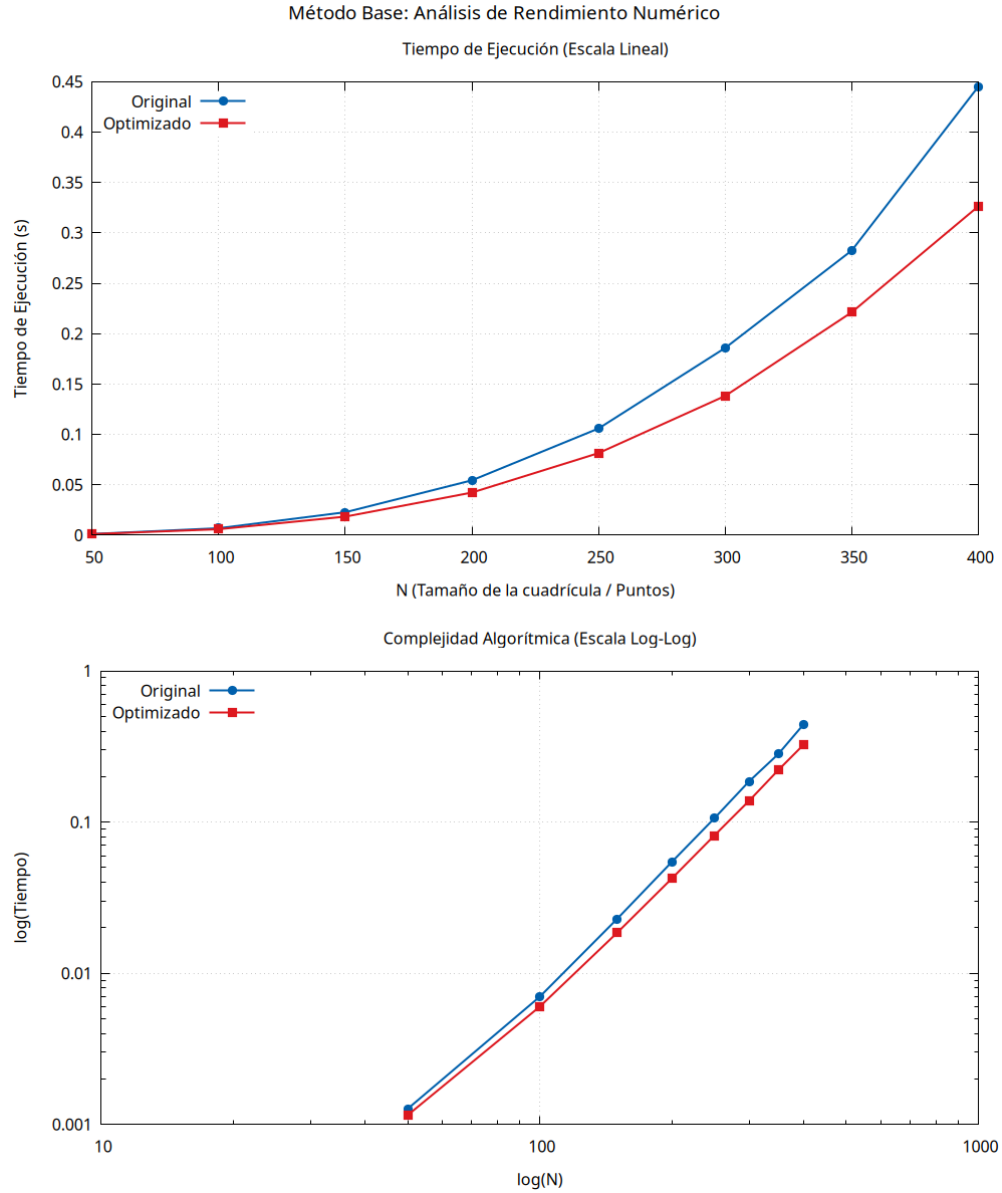


Figura 12: Resumen comparativo del Speedup serial alcanzado en los diferentes métodos numéricos.

Cuadro 5: Resumen de métricas de optimización serial.

| Método   | Mejora Relativa (Speedup)  | Factor Limitante Principal     |
|----------|----------------------------|--------------------------------|
| FDM      | $> 100\times$ (Asintótico) | Cache L3 / Ancho de Banda      |
| Sinc-DVR | $1,8 \times - 2,2 \times$  | Diagonalización densa (LAPACK) |
| Shooting | $3,5 \times$               | Evaluación del potencial       |

La evidencia experimental sugiere que el uso de banderas de optimización (`-O3 -march=native`) es insuficiente sin garantizar la unidad de los saltos de memoria (*stride-1*). Estos resultados validan la robustez de las implementaciones seriales para proceder con la distribución de carga mediante OpenMP.

## 5. Paralelización con OpenMP

### 5.1. Análisis de Escalabilidad y Speedup

- **Speedup Superlineal:** Observado en Lanczos ( $E_p > 1,0$ ), producto de la división del dominio que permite que los subconjuntos de datos quepan íntegramente en la caché rápida de cada núcleo.
- **Ley de Amdahl:** El método de Cayley presentó un techo de escalabilidad ( $S \approx 1,16$ ) debido a las dependencias de datos secuenciales en la solución del sistema tridiagonal.

## 6. Conclusiones

La eficiencia en HPC no es solo una cuestión de hilos, sino de la simbiosis entre el modelo físico y la arquitectura. El Método de Lanczos emerge como la solución óptima para este problema debido a su localidad de datos y escalabilidad excepcional.